

Mini-LSM v1

This is a legacy version of the Mini-LSM course and we will not maintain it anymore. We now have a new version of this course. We keep the legacy version in this book so that the search engine can keep the pages in the index and users can follow the links to the new version of the course.

V1 Course Overview



In this course, we will build the LSM tree structure in 7 days:

- Day 1: Block encoding. SSTs are composed of multiple data blocks. We will implement the block encoding.
- Day 2: SST encoding.
- Day 3: MemTable and Merge Iterators.
- Day 4: Block cache and Engine. To reduce disk I/O and maximize performance, we will use moka-rs to build a block cache for the LSM tree. In this day we will get a functional (but not persistent) key-value engine with `get`, `put`, `scan`, `delete` API.
- Day 5: Compaction. Now it's time to maintain a leveled structure for SSTs.
- Day 6: Recovery. We will implement WAL and manifest so that the engine can recover after restart.
- Day 7: Bloom filter and key compression. They are widely-used optimizations in LSM tree structures.

Development Guide

We provide you starter code (see `mini-lsm-starter` crate), where we simply replace all function body with `unimplemented!()`. You can start your project based on this starter code. We provide test cases, but they are very simple. We recommend you to think carefully about your implementation and write test cases by yourself.

- You can use `cargo x scheck` to run all test cases and do style check in your codebase.
- You can use `cargo x copy-test dayX` to copy test cases to the starter code.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.